



Semáforos con Arduino: Un Enfoque Práctico en Programación

Proyecto docente para la asignatura Fundamentos de la Programación.
Implementación práctica utilizando Code::Blocks (C) y una placa Arduino para modelar el cambio de estados de un semáforo y su lógica de control.

Introducción a la Programación y a los Semáforos

Breve repaso de conceptos fundamentales: variables, estructuras de control, funciones, temporización y manejo de E/S digital. Se contextualiza el problema del control de semáforos como ejercicio integrador que combina teoría en C (Code::Blocks) con práctica en hardware (Arduino).



Hardware

Arduino UNO: pines digitales, salidas LED y temporizadores.



Software

Programación en C usando Code::Blocks; estructura modular y pruebas unitarias básicas.



Concepto

Estados: Rojo, Amarillo, Verde. Transiciones seguras y temporizaciones.



Planteamiento del problema

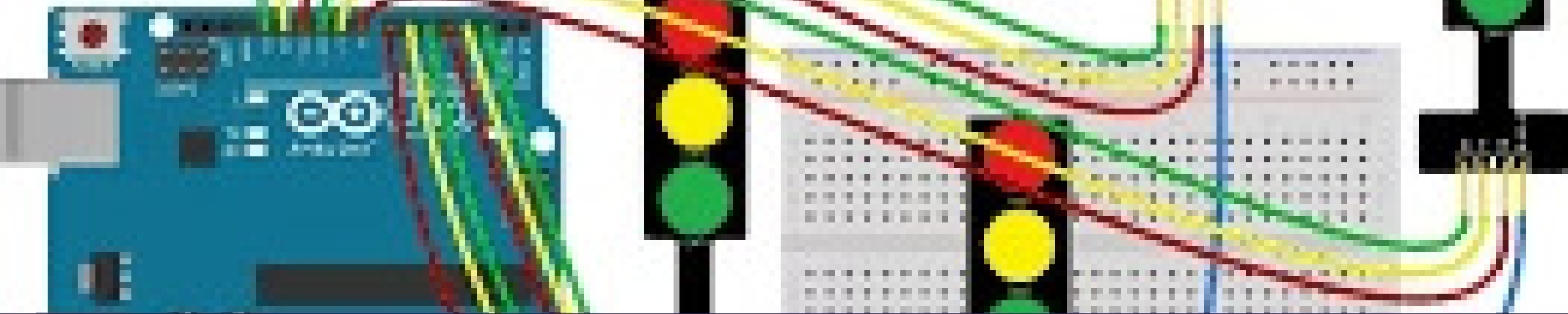
3.1 Formulación del problema

¿Cómo diseñar e implementar un controlador de semáforo con Arduino que permita cambiar de estado de forma segura y predecible, ajustando tiempos y respondiendo a entradas simples (por ejemplo, sensor o botón) para simular condiciones reales de tráfico?

3.2 Justificación

El desarrollo fortalece competencias clave en programación estructurada (C), depuración en IDE (Code::Blocks), y en la integración hardware-software con Arduino. Tiene aplicación directa en sistemas embebidos, automatización y toma de decisiones en tiempo real para control de tráfico a pequeña escala.





Sistema de Objetivos

Objetivo general

Diseñar, programar y verificar un prototipo funcional de semáforo usando C en Code::Blocks y una placa Arduino que gestione transiciones temporizadas y permita intervención manual.

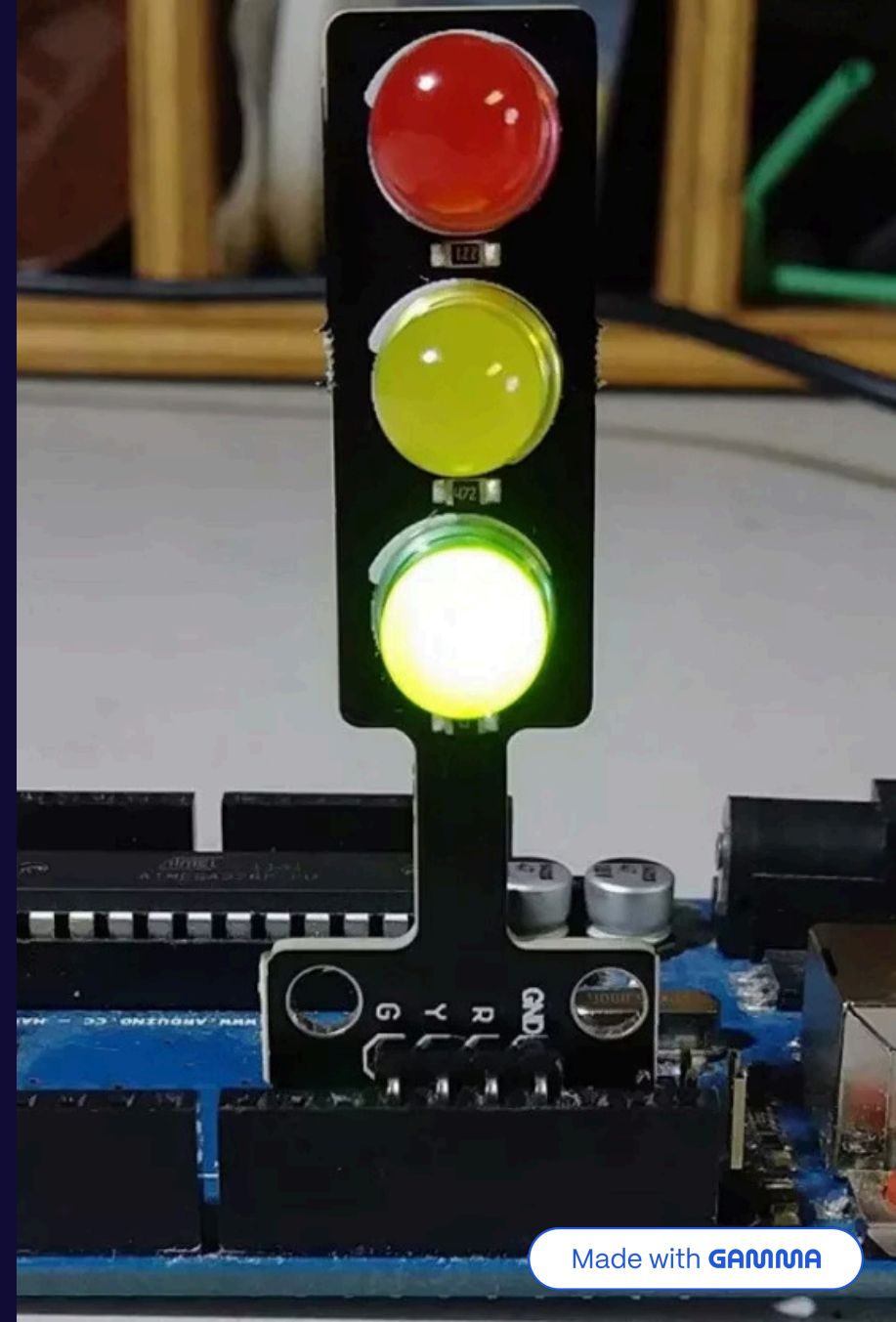
Objetivos específicos

- Implementar la lógica de estados (Rojo → Verde → Amarillo → Rojo) en C, modularizada en funciones.
- Integrar temporizadores y entradas digitales en Arduino para controlar el cambio de luces y permitir una señal de prioridad (botón/sensor).
- Documentar y validar el comportamiento mediante pruebas y una demostración práctica que verifique tiempos y transiciones seguras.

Alcance

El proyecto se limita a un prototipo de semáforo de una intersección simple (tres LEDs: rojo, amarillo, verde). No incluye control de múltiples intersecciones ni detección avanzada de flujo vehicular. Se espera: diseño del código, montaje en protoboard, pruebas funcionales y demostración en laboratorio.

- ① Limitaciones: no se modelan peatonales ni comunicación V2X; tiempos son configurables pero no adaptativos al tráfico real.



Metodología — Marco de trabajo 5W + 2H

What: Prototipo de semáforo (hardware + software).

Why: Practicar conceptos de programación en C y control embebido; relevancia educativa y aplicabilidad en automatización.

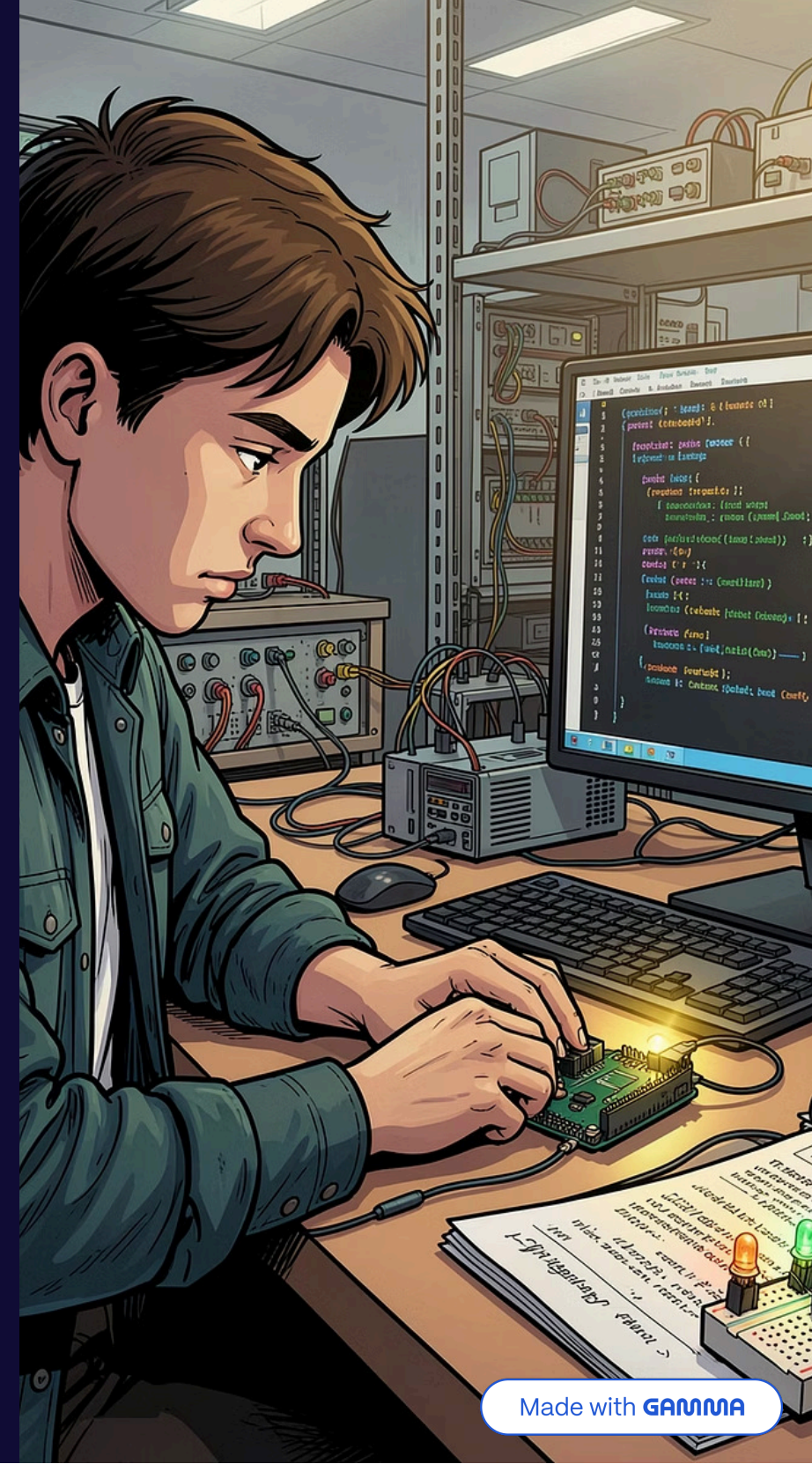
Who: Estudiantes de Fundamentos de la Programación; supervisor docente.

When: Cronograma de laboratorio (4 semanas: diseño, implementación, pruebas y demostración).

Where: Laboratorio de electrónica / aula de programación.

How: Desarrollo iterativo: 1) especificación de estados, 2) codificación en C, 3) pruebas en simulador/Arduino, 4) ajuste de temporizaciones.

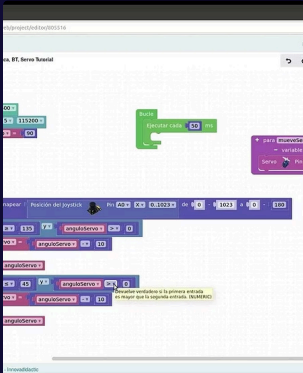
How much: Recursos mínimos — Arduino UNO, 3 LEDs, resistencias, protoboard, cables; tiempo estimado: 20–30 horas por grupo.



Desarrollo del Código (Code::Blocks, C) y Simulación (Arduino)

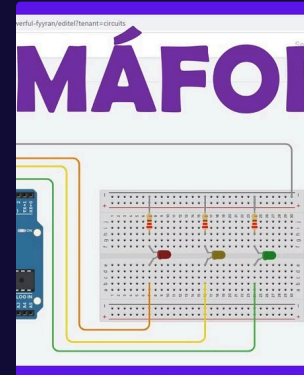
Descripción técnica concisa del enfoque de implementación:

1. Arquitectura: main() inicializa pines, luego ciclo principal que invoca función stateMachine().
2. Módulos: initGPIO(), setState(enum), delayMillis(uint32_t) — separación de responsabilidad para pruebas.
3. Temporización: uso de millis() en Arduino para evitar bloqueos y permitir lecturas de entrada (botón) durante los intervalos.



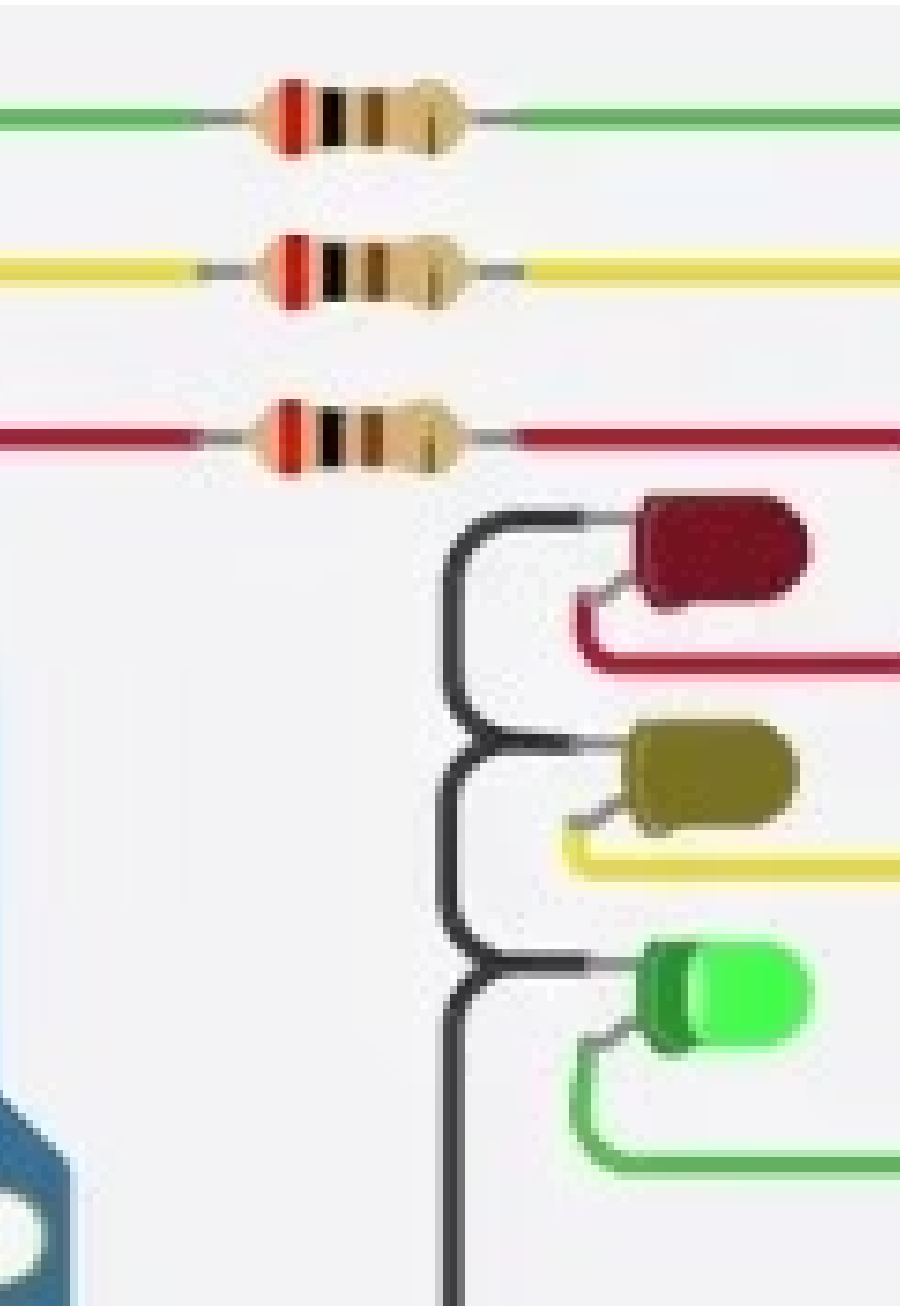
Ejemplo (esquema)

`void setState(int state) { /*
activar/desactivar pines */ } — lógica
simple y testeable. Separar hardware-
abstracción para facilitar pruebas en
PC.`



Simulación

Se recomienda validar primero en
Tinkercad (link de recursos del curso),
luego subir el sketch al Arduino para la
prueba física.



Demostración Práctica del Semáforo en Acción

Protocolo de demostración:

- Arranque: LEDs inicializan en Rojo por seguridad.
- Secuencia automática: Rojo (5s) → Verde (4s) → Amarillo (2s) → Rojo.
- Intervención: pulsador simula sensor; si se presiona durante Rojo, se prioriza cambio seguro (espera a fin de ciclo).

❏ Sugerencia: grabar video corto de la simulación en Tinkercad y anexarlo a la entrega práctica.


```
0; // the red LED is connected to pin 13
pinMode(led_red, OUTPUT);
pinMode(led_yellow, OUTPUT);
pinMode(led_green, OUTPUT);
```

```
1 the LEDs as OUTPUT
pinMode(led_red, OUTPUT);
pinMode(led_yellow, OUTPUT);
pinMode(led_green, OUTPUT);
```

```
2 green LED on and the other LEDs off
digitalWrite(led_red, LOW);
digitalWrite(led_yellow, LOW);
digitalWrite(led_green, HIGH);
delay(2000); // wait 2 seconds
```

```
3 yellow LED on and the other LEDs off
digitalWrite(led_red, LOW);
digitalWrite(led_yellow, HIGH);
digitalWrite(led_green, LOW);
delay(1000); // wait 1 second
```

```
4 red LED on and the other LEDs off
digitalWrite(led_red, HIGH);
```

Conclusiones y Reflexiones

El ejercicio integra conceptos teóricos y habilidades prácticas: programación en C, modularización, manejo no bloqueante del tiempo en Arduino y pruebas en simulador. Fortalece la capacidad de diseñar sistemas embebidos sencillos y documentar resultados.

Lecciones aprendidas

- Importancia de separar lógica de control y acceso hardware.
- Uso de `millis()` evita bloqueos y facilita respuesta a eventos.
- Pruebas incrementales (simulación → hardware) reducen errores.

Próximos pasos

- Extender a control de intersecciones múltiples.
- Incorporar detección de flujo (sensores ultrasonidos).
- Evaluación de adaptatividad temporal basada en tráfico.

Bibliografía

Fuentes académicas y recursos consultados (Google Scholar / Zotero):

- Rajko, M.; "Embedded Systems for Traffic Control: A Review", Journal of Embedded Systems, 2018. (Disponible en Google Scholar)
- Sánchez, A.; "Control lógico de semáforos con microcontroladores", Revista de Automatización, 2019. (Revisión académica en Google Scholar)
- Monroy, L.; "Programación en C para sistemas embebidos", Editorial Técnica, 2020. (Referencia didáctica citada en Zotero)
- Arduino Documentation and Examples — Arduino.cc (referencia técnica para funciones como millis(), pinMode(), digitalWrite()).
- Simulación y esquemas de montaje: Proyecto y componentes en Tinkercad — recurso compartido por el docente: <https://www.tinkercad.com/things/bWT6vevRdUR/editel?lessonid=EFU6PEHIXGFUR1J&projectid=OGK4Q7VL20FZRV9&collectionid=undefined&title=Editing%20Components#/lesson-viewer>

Nota: Las referencias bibliográficas completas (DOI, autores y fechas) se obtuvieron y registraron mediante búsquedas en Google Scholar y la biblioteca Zotero del curso; incluir en la memoria final según formato APA o IEEE requerido por la asignatura.